

CULPRIT

The AI debugger that proves the bug,
then proves the fix.

Live probe → failing test → root cause → tested fix → commit

Built with IBM Bob IDE · Runs on IBM Bob Shell · watsonx Orchestrate agent

github.com/Poornasri2003/culprit

The problem: silent bugs in running services

- Developers spend 35-50% of their time validating and debugging (ACM Queue)
- The costliest bugs live in production: the service answers 200 OK, but wrong
- No crash, no stack trace, tests green; the cause often sits in another codebase
- Fixing by hand: reproduce, read several repos, fix, prove, add a test (hours)
- Code-only or log-only AI assistants guess; they never check the live behaviour

Example (our demo):

```
$ curl -X POST /cart/total -d '{"items":[{"price":20,"qty":2}], "discount_code":"SAVE10"}'  
{"total": 32.4}      <- documented rule says 33.2
```

The solution: Culprit

Give it a running service, the code behind it, and one sentence about the bug.

Observe

calls the live service
and records what it does

Prove

IBM Bob writes a test
that fails on the bug

Explain

IBM Bob traces the root
cause across codebases

Fix

IBM Bob proposes ranked
minimal fixes

Verify

whole test suite must pass,
else revert and retry

Deliver

commit fix + regression
test, with a report

Who it's for: backend, platform and on-call engineers of multi-service systems.

How: one CLI command, an HTTP API, or a watsonx Orchestrate chat agent.

Demo: wrong answer to committed fix in 63 s

```
$ culprit debug --url http://localhost:8000/cart/total --method POST --body ... \  
  --auth-bearer --folder frontend --folder backend --folder shared \  
  --issue "cart total wrong when discount code applied"  
0.3s   probe live URL           HTTP 200 {"total":32.4}  
0.3s   Reproducer || CauseTracer (2 IBM Bob subagents in parallel)  
37.7s  repro test FAILS on current code -> bug proven  
38.6s  FixAuthor: 3 ranked fixes (exact edits)  
47.5s  Guard: permanent regression test  
62.9s  all tests PASS -> commit  
Status: FIXED   63s   0.21 Bobcoins   root cause shared/pricing.py:28
```

Restart the service: {"total": 33.2} (correct)

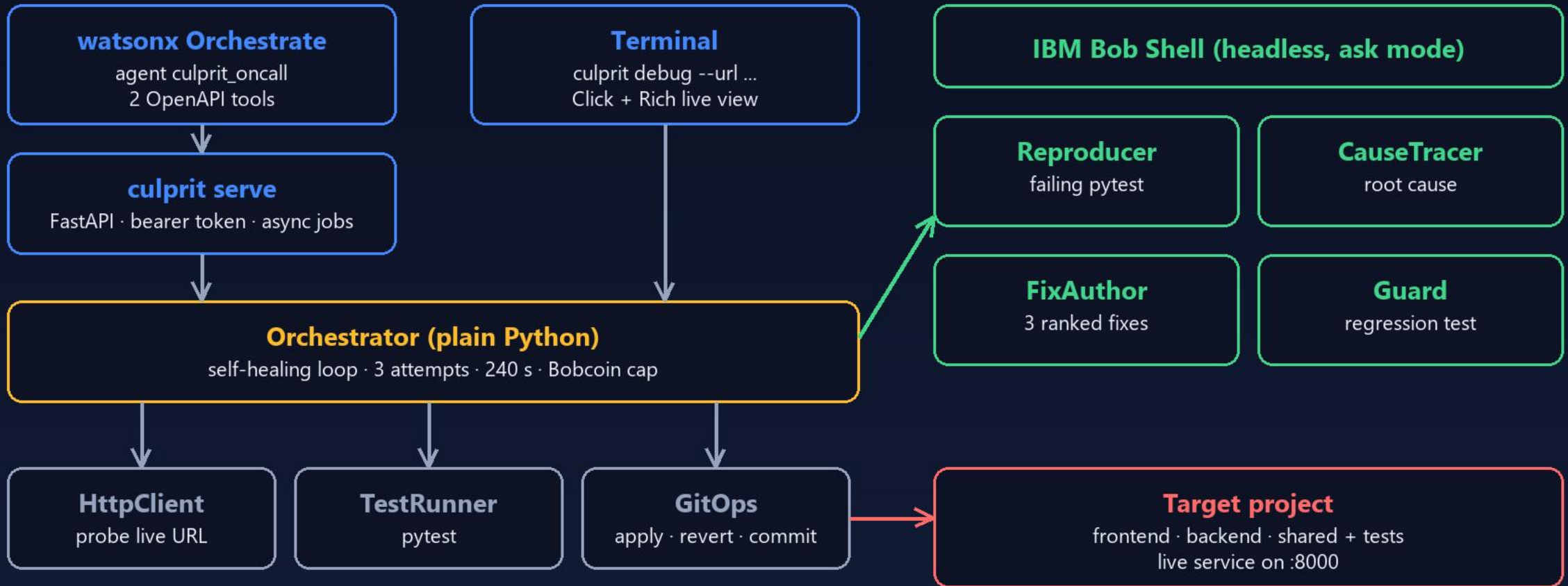
Flow: from bug report to commit



Gates (decided by code, never by AI):

- Repro test passes on the current code -> NEEDS_HUMAN (bug not proven)
- Confidence below 0.70 -> NEEDS_HUMAN
- Tests fail after the fix -> revert files, retry with the failure as context (max 3)
- Bobcoin budget used up -> PARTIAL

Architecture



Agents, subagents and tools

Entry points

Terminal: culprit debug
Orchestrate agent
culprit_oncall (chat)

Orchestrator

Plain Python, no AI
3 attempts, 240 s,
Bobcoin cap

Tools (code)

HTTP probe · pytest
git apply / revert
/ commit

IBM Bob subagents (Bob Shell, headless, read-only ask mode)

Reproducer → pytest that must FAIL on the current code
CauseTracer → file, line, explanation, confidence (runs in parallel with Reproducer)
FixAuthor → up to 3 fixes, lowest risk first, as exact search/replace edits
Guard → permanent regression test in the project's own style

The test suite is the judge: nothing is committed unless every test passes.

How we used IBM Bob

1. Built with Bob IDE

SPEC.md first, then 7 Bob prompts, one architecture layer at a time:

- Plan mode: ARCHITECTURE.md
- Agent mode: the demo app with the bug
- models, infrastructure, Bob client, 4 subagents, orchestrator, CLI
- Every task exported to bob_sessions/
- Reviewed + verified with real Bob runs
- About 19 of 40 Bobcoins used

2. Bob inside Culprit

All 4 subagents are IBM Bob, via Bob Shell (headless):

```
bob run --format json --mode ask  
      --max-cost <remaining budget>
```

- Reads code across all codebases
- Read-only: Culprit applies the edits
- Bobcoin cap enforced twice
- About 0.2 Bobcoins per full fix
- Keys never logged, headers redacted

Chat-triggered debugging: watsonx Orchestrate

- Agent culprit_oncall lets an on-call engineer fix a bug from chat
- Two OpenAPI tools: start_culprit_debug(issue), get_culprit_debug_result(job_id)
- They call culprit serve: a bearer-token-protected FastAPI service
- Asynchronous by design: a run (about 60 s) exceeds Orchestrate's 40 s tool limit
- Callers choose only the issue text; the project and URL are fixed server-side
- Tool endpoints tested end to end over a public tunnel: FIXED in 73 s
- One command deploys the agent: scripts/orchestrate_deploy.py

```
Orchestrate chat -> agent -> tools -> tunnel -> culprit serve -> Bob Shell
```

Tech stack

IBM

IBM Bob IDE (Plan + Agent mode)
IBM Bob Shell 2.0.5 (headless)
watsonx Orchestrate (agent,
OpenAPI tools, ADK 2.17)

Core (Python 3.11+)

Click · Rich (CLI + live view)
Pydantic v2 (domain models)
anyio (parallel subagents)
httpx (live probe)
GitPython (commit / revert)
pytest (runner + 127 tests)

Service & demo

FastAPI + uvicorn (culprit serve)
Cloudflare tunnel
Flask (demo shop services)
python-dotenv (.env secrets)

Review: Claude Code · License: MIT · github.com/Poornasri2003/culprit

Why it's different, and why it matters

Proof, not guesses

Bug must be reproduced by a failing assertion before any fix
Fix is kept only if every test passes; otherwise files are restored exactly

Runtime + code evidence

Probes the live service, then traces across several codebases
Finds bugs that return 200 OK with the wrong value

Business value

About 1 hour of debugging -> about 60 s
Every fix ships with a regression test
Costs about 0.2 Bobcoins per bug

Safe by design

Bob is read-only; code applies edits
Hard Bobcoin budget per run
Secrets only in .env, redacted

Results and what's next

FIXED

live demo bug

~60 s

bug to commit

0.21

Bobcoins per run

127

unit tests

Next steps

- Clone codebases from --git; JavaScript/TypeScript test runners
- Open a GitHub pull request instead of a local commit
- Deploy the Orchestrate agent for team-wide, chat-first on-call

github.com/Poornasri2003/culprit

Thank you!